BEFORE YOUR AGENT READS ONE WORD OF YOUR QUESTION

1,170,523

tokens to describe **2,594** tools as plain MCP schemas

585% of a 200K context window

```
same API through Code Mode: ~1,000 tokens
```

Cloudflare's own API · github.com/cloudflare/mcp

💬 How many MCP servers is your agent connected to? Type a number.

From 150K to 22K tokens

Code Mode and the context cost of MCP tools



Nishant Thakur · Adobe
linkedin.com/in/nishantkthakur



Sonam Raj · Adobe
linkedin.com/in/sonamraj04

What you get from this talk

- A pattern: show the model tools as code, not schemas.
- A measurement you can run on your own agent.
- A command to try tonight.

Plus the places it broke.

Our agent, before the first question



150K

of a 200K window, as tool definitions

Outlook	**24K**
SharePoint	**20K**
Jira	**18K**
Confluence	**14K**
Slack	**22K**
Other dev tools	**52K**

75% gone before "hello".

1 block = 2K tokens · totals measured, per-server split estimated

Failure 1: the schema dump

```
gdrive.getDocument
  Description: Retrieves a document from Google Drive
  Parameters: documentId (required, string) …
  Returns: Document object with title, body, metadata …
```

```
salesforce.updateRecord
  Description: Updates a record in Salesforce
  Parameters: objectType, recordId, data (required) …
  Returns: Updated record with confirmation
```

anthropic.com/engineering/code-execution-with-mcp

Multiplied by every tool, on every request.

Failure 2: results go through the model twice

```
TOOL CALL: gdrive.getDocument(documentId: "abc123")
  → [full transcript text]  (into model context)
```

```
TOOL CALL: salesforce.updateRecord(
  data: { Notes: [full transcript, typed out again] } )
```

2-hour meeting: ~50,000 extra tokens

anthropic.com/engineering/code-execution-with-mcp

Picking the wrong tool

42%

OF THE TIME THE MODEL PICKED THE RIGHT TOOL

> "The most common failures are wrong tool selection and incorrect parameters, especially when tools have similar names."
>
> `anthropic.com/engineering/advanced-tool-use`

Our agent has many MCPs with look-alike tools: `search`, `get` and `list` everywhere.

PART 1

Our bet: LLMs are better at writing code than at calling tools

Why code works better

Tool call
special tokens + JSON

Real code
all over the training data

typed API
instead of tool calls → model writes code → secure sandbox
runs it → only the results it needs
come back

Cloudflare
Sep 2025 · calls it Code Mode → **Anthropic**
Nov 2025 · similar approach, credits Cloudflare

We demo an agent using it later.

QUICK GUESS

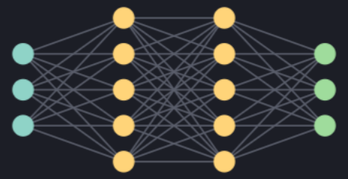
Drive transcript to Salesforce: which option sends less through the model?

A
2 tool calls,
every result returned

B
1 script,
only the summary returned

💬 A or B?

The same task as code fixes failure 2



Model
writes the script, sees only the result

↓ script

the transcript never goes up here

↑ short result

SANDBOX

1. import the tools
   as a library

→

2. `gdrive.getDocument()`
   saved in a variable
   `transcript`

`transcript` →

3. `salesforce.updateRecord()`
   variable passed straight through

`tool calls · 150K tokens`

`code · 2K tokens  98.7% less (Anthropic's example)`

The model finds tools the way a developer would

fixes failure 1

```
servers/
├── google-drive/
│   ├── getDocument.ts
│   └── index.ts
├── salesforce/
│   ├── updateRecord.ts
│   └── index.ts
└── …
```

```ts
/* Read a document from Google Drive */
export async function getDocument(
  input: { documentId: string }
): Promise<{ content: string }> {
  return callMCPTool(
    'google_drive__get_document', input);
}
```

Anthropic calls this progressive disclosure: list the directory, read only the file you need, and pay only for the tools you use.

How we measured

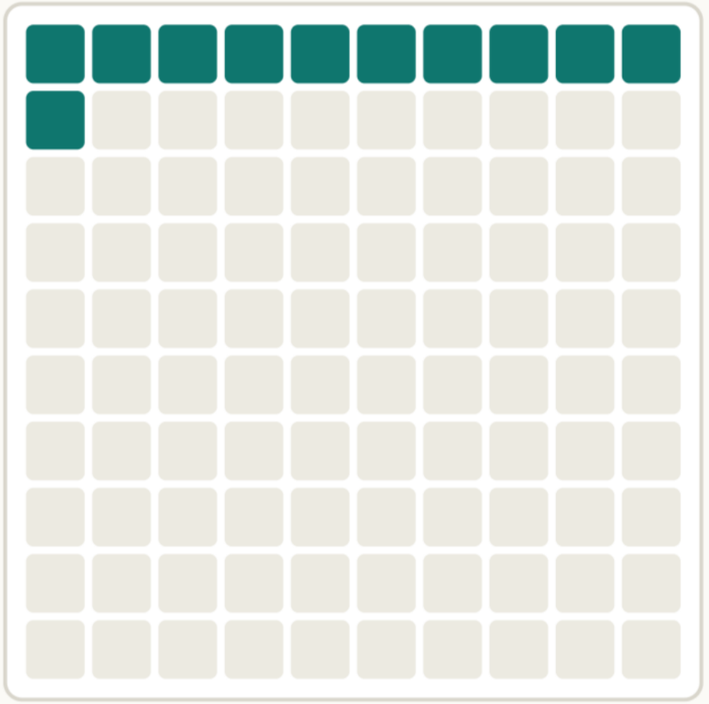
One agent, connected to the usual developer MCPs:

Outlook SharePoint Jira Confluence Slack + more

Measured before (direct tool calls) and after (Code Mode: one typed stub per tool). Same tasks, same model.

1	Prompt tokens before the user's question	log it on every request
2	Right tool picked, %	on a labelled task set

Same agent, after Code Mode



22K

code tool + index + stubs loaded on demand

22K. Same tools, 85% less.

42% → 80%

RIGHT TOOL PICKED

1 block = 2K tokens · 22K = prompt tokens before the question (the demo counts results too) · per-server split estimated

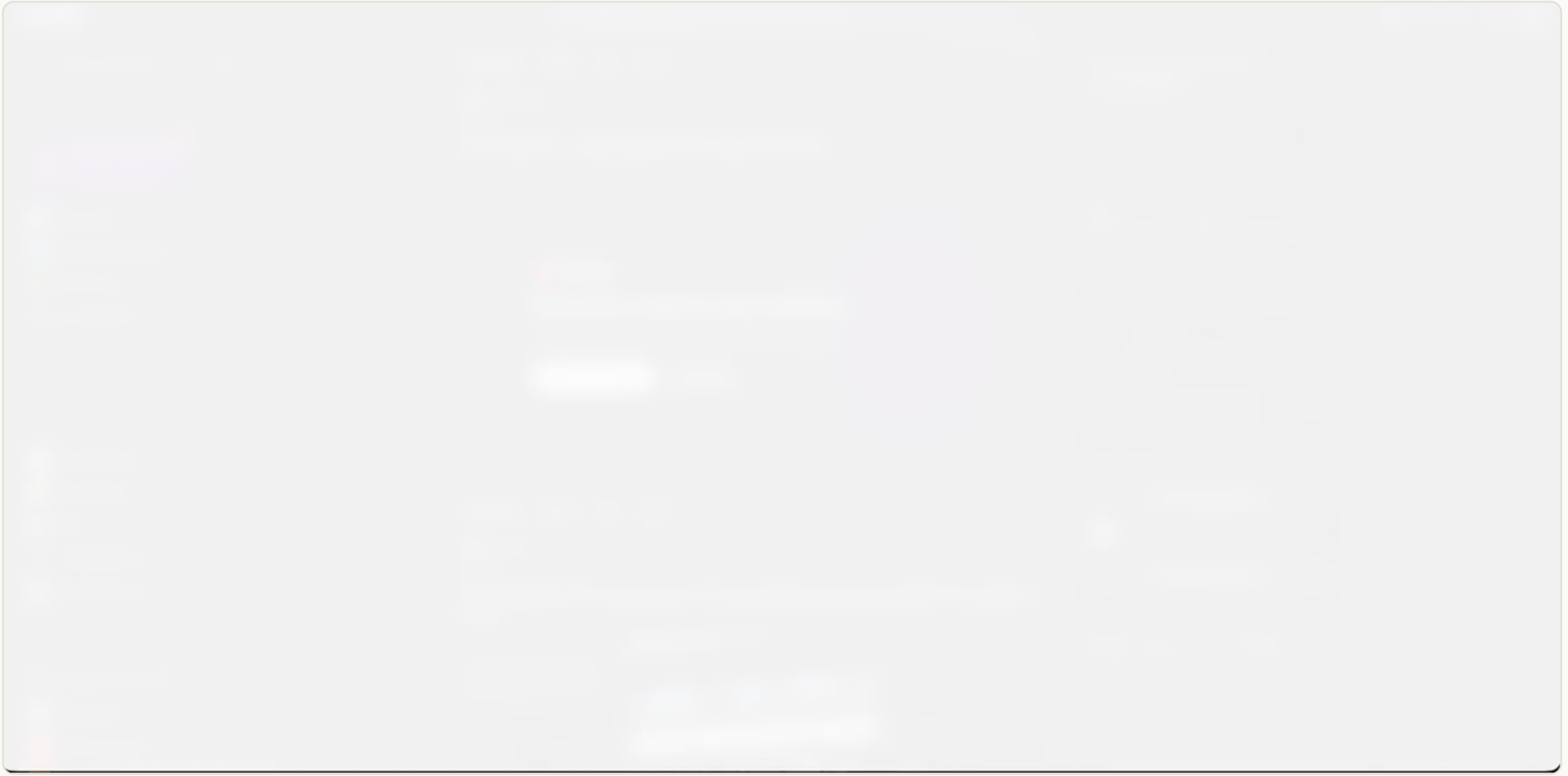
PART 2

Demo: the same task in pi, with tools and with code

Our production agent is our own. For the demo we use pi, which ships Code Mode, with dummy MCP servers. Any agent with a sandbox can do this.

Round 1: plain tool calling

86 tools (dummy Jira, Confluence, Slack MCPs) · 10 calls · **54K definitions + 42K results ≈ 98K**



BEFORE THE REVEAL

Same task with `codemode`. How many tokens?

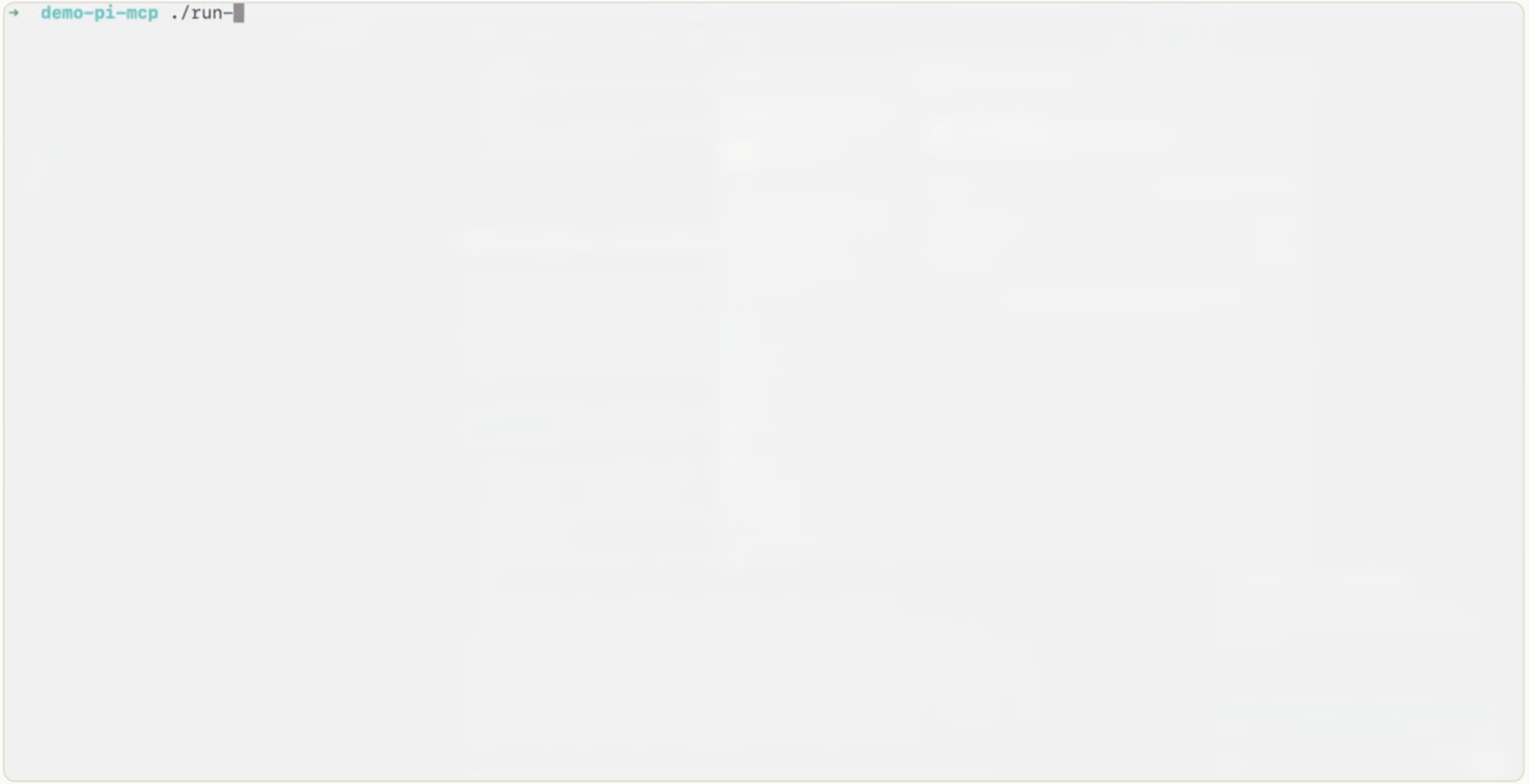
💬 Type your guess now.

34,000

tokens in context · 10 scripts · 65% less than Round 1

Round 2: codemode

1 tool declared · 10 scripts · **≈ 34K in context** (Round 1: 98K)



The script decides what comes back

```
const [i440, i447, i414, p1, p2] = await Promise.all([
  tools.mcp__jira__get_issue({ issueKey: "AUTH-440" }),
  tools.mcp__jira__get_issue({ issueKey: "AUTH-447" }),
  tools.mcp__jira__get_issue({ issueKey: "AUTH-414" }),
  tools.mcp__confluence__get_page({ pageId: "48213", bodyFormat: "view" }),
  tools.mcp__confluence__get_page({ pageId: "48977", bodyFormat: "view" }),
]);
function pick(r) { return r.structuredContent ?? r.content; }
const issues = [i440, i447, i414].map(pick);
const out = { issues: issues.map(x => ({ key: x.key, summary: x.summary,
  priority: x.priority, status: x.status, assignee: x.assignee, url: x.url,
  desc: String(x.description || "").slice(0, 400) })) };
const s1 = pick(p1), s2 = pick(p2);
out.p1 = { title: s1.title, url: s1.url, bodyPreview: JSON.stringify(s1).slice(0, 500) };
return out;
```

script 7 of 10, trimmed to fit · 5 MCP calls in parallel

How pi runs it



MCP tools stay out of the prompt. The model finds them with `searchTools()` when it needs them. It is the same idea as the directory listing, with search instead.

```
pi docs/codemode.md
```

PART 3

Where it bit us

Failure modes we hit

Symptom	Fix
Model calls a method that doesn't exist	Sandbox error names the nearest real one. Retry.
Scripts break after a server upgrade	Regenerate stubs at connect time, never commit them.
Token savings vanish	One file per tool, not one big `index.d.ts`.
Model prints raw results and tool descriptions	Tell it to return only the fields it needs. (Our takes: 45K, then 34K with the hint.)
"Why did it do that?"	Log every `tools.*` call in the sandbox.

When not to use it

- You have 3 tools and simple tasks. A sandbox isn't worth it.
- You don't have a sandbox you trust.
- Every step needs human approval, and code batches steps.

A simpler option is tool search. (`pi --tools +tool_search`)

💬 What would block this at your work?

Try it tonight

1. Measure the prompt tokens before the user's question.
2. Turn it on with `pi --tools +codemode`, a server-side Code Mode, or your own stubs.
3. Measure again with the same tasks and the same model.

1.17M → 1K · 150K → 2K · 150K → 22K Cloudflare · Anthropic · ours

Thanks



Nishant Thakur

linkedin.com/in/nishantkthakur



Sonam Raj

linkedin.com/in/sonamraj04

💬 Tools or code: which does your next agent use?